SLIDE 01 / 09

Two problems

PRESENTER CUES · NOT SPOKEN

- Two changes: delivery system and runtime judgment
- Present diagnosis, with organisational rebalancing
- Small shared-context teams are an example, not a threshold

SPOKEN SCRIPT · 132 WORDS

Hello everyone. I’m Vitalii. I’ve worked in IT for over twenty years, across development, testing and delivery. I built PMO and Delivery foundations in my organisation and use AI alongside my team.

Writing and printing moved memory outside our heads. Ideas could travel, outlive their authors and transform society. AI lets us delegate part of problem-solving itself. That scale of change requires precise problem statements.

My first diagnosis is that AI adoption has pushed the SDLC out of balance at industry level. Faster individual work changes demands on the whole delivery system. Organisations need deliberate rebalancing.

Second: when models construct solutions at runtime, we need to engineer control around that judgment in its business context.

Small teams sharing context and roles may coordinate more easily. Narrow, low-risk delegation may need lighter controls.

SLIDE 02 / 09

The whole delivery system

PRESENTER CUES · NOT SPOKEN

- Illustrative comparable changes per month: 8, 100, +92
- Generation differs from admission; drafts can be discarded
- Possible constraints: downstream capacity, useful demand, user adoption

SPOKEN SCRIPT · 177 WORDS

Every stage can handle eight comparable changes a month. Coding capacity jumps to a hundred. Downstream capacity remains eight.

If we admit all hundred, waiting work grows by ninety-two a month. Checks may weaken. If poorer changes reach production, repair can consume the gains.

Generated drafts do not all have to become accepted work. We can discard them. Admission limits keep internal WIP bounded, while excess demand waits outside.

Suppose we automate review. Testing or integration may become the next constraint.

Suppose all downstream stages can handle a hundred. Product, business, sales and marketing must still discover and validate enough useful work. Useful demand determines the work worth delivering.

Users also need time to discover features, learn them and change their habits. Even useful features can overwhelm their capacity to absorb change.

These are possible constraints. Their order depends on the organisation.

The target is useful change at sustainable total cost. We can increase capacity, reduce incoming work or choose a different scope.

And the system includes how people learn and recover. Code review makes that visible.

SLIDE 03 / 09

Review and ownership

PRESENTER CUES · NOT SPOKEN

- Two functions: verification evidence and shared learning
- Accountability alone does not create operational competence
- Preserve flow and learning through a working replacement

SPOKEN SCRIPT · 127 WORDS

Code review produces verification evidence. It also transfers knowledge.

During review, people question decisions and connect changes to the rest of the system. That shared understanding supports competent ownership.

Assigning accountability does not give someone the knowledge needed to act.

The comprehension gap appears when accepted changes outpace the team’s ability to understand what it owns.

Delegating code does not delegate responsibility. Automating checking can remove a queue and part of the learning process together.

A replacement workflow must demonstrate both adequate checks and enough shared understanding to explain, maintain, diagnose and recover the system.

This does not require everyone to read every line. We need a learning mechanism that actually works.

Human learning also constrains design, testing and operations. Rebalancing must protect flow and learning together.

The evidence

PRESENTER CUES · NOT SPOKEN

- Different samples and methods; no common measured pipeline
- November Chen–Stratton: 655 firms and corrected endpoints
- Mechanism evidence differs from industry-level synthesis

SPOKEN SCRIPT · 117 WORDS

These reports measure different populations and stages.

NBER estimates about twenty-five times as many changed lines, three point four times as many commits, but about thirty percent more releases. Different samples and tool stages mean this is not one measured pipeline.

The November Chen–Stratton revision covers 655 firms. Agent adoption increased opened PRs by 35.7 percent, against 13.7 percent more resolved Jira issues, with the latter borderline significant. Submission-to-approval elapsed time rose 56.1 percent. That includes waiting, not just reviewer work.

Other studies report instability, greater complexity and maintainability concerns. App supply also outpaced early-use signals.

These findings support parts of the mechanism. My industry-level diagnosis is a systems synthesis. An intervention’s business advantage still needs demonstration.

SLIDE 05 / 09

Finding a new balance

PRESENTER CUES · NOT SPOKEN

- Extensive delegation, limited assistance or no AI
- Owner needs authority, budget and cross-functional support
- Bounded pilot and safe recovery drill; compare before/during and keep scope limits

SPOKEN SCRIPT · 233 WORDS

Treat the six approaches here as hypotheses.

Agentic review may scale checking, but models can share blind spots. Who learns?

Living documentation may help. Who verifies the description, and can people use it to recover the real system?

There is no universal AI-native recipe. Depending on the work, domain, criticality, capabilities and economics, the right operating model may use extensive delegation, limited assistance or no AI.

An enterprise’s adjustment may outlast its investment horizon. Waiting, rework and tool costs keep accumulating.

Give an accountable owner the authority, budget and cross-functional support to rebalance the system.

Then run a bounded pilot. Define the baseline, comparable work, useful outcomes, full delivery and operating cost, workload limits and stop conditions.

Include a recovery drill in a safe test environment. An engineer who did not author the change uses the normal on-call tools, access and documentation. Can they explain the behaviour, diagnose a plausible failure and restore service within the business-defined time?

Compare equivalent scenarios before and during the pilot. A passed drill supports only the scope tested. Investigate failures. Faster review cannot justify an unacceptable loss of operational capability.

If the budget is small, narrow the pilot. Listen to the team, especially the sceptics. Their observations can expose costs that dashboards miss.

Keep interventions that improve useful delivery at acceptable total cost while preserving the ability to own the system. Change or stop those that fail.

SLIDE 06 / 09

Decisions inside the product

PRESENTER CUES · NOT SPOKEN

- Explicit rules versus runtime model judgment
- Representative cases and repeated trials
- Readiness can mean research; completion needs controls and fallback

SPOKEN SCRIPT · 174 WORDS

Now to projects that delegate solution-building to a model.

In conventional development, we specify the intended behavior and implement the rules that produce it.

With delegated model judgment, the model chooses the solution path at runtime. Requirements must define an acceptable region across many possible paths: correct outcomes, allowed actions and conditions for stopping or asking for help.

We specify what must remain true while the model chooses how to proceed.

Testing changes too. One failing test can expose a defect. A passing example tells us little about behavior across that space. We need representative cases, repeated trials and evidence about failure frequency and severity. That takes time and expertise.

Readiness changes. Clear requirements may be enough to start conventional implementation. Here, we may first need research to establish whether we can evaluate and control the intended behavior.

Done requires behavior evidence, working controls and tested fallback.

For DevOps, uncertainty now sits inside the application’s decision-making core. Control effort must match the delegation and consequences.

I believe these systems deserve a distinct engineering category.

Control feasibility before delivery

PRESENTER CUES · NOT SPOKEN

- A demo leaves many paths untested
- Complete control loop before production commitment
- Human availability and capacity are architectural dependencies

SPOKEN SCRIPT · 183 WORDS

This changes project initiation.

A proof of concept can establish feasibility and collect feedback. It does not prove production readiness.

An agentic demo can hide a much larger behavior space. A few users see the happy path. Different users, longer interactions or changing conditions can expose hidden failures.

So we must establish whether we can control the intended delegation.

Design the whole control system: permitted actions, enforced constraints, signals that reveal problems, and a mechanism that intervenes and checks the result.

The business context determines how strict those controls must be.

Establishing feasibility may itself require expensive research. That work should reveal risks, required expertise and operating costs before we commit to production.

We may need a domain specialist, control expertise or a human operator inside the loop.

If we depend on that person to detect problems or authorize actions, their availability and capacity become architectural dependencies.

This planning gives us a better basis for decisions. We may reduce delegation, narrow the use case or stop.

We must plan and fund the complete control system. A convincing demo alone cannot establish its viability.

SLIDE 08 / 09

Service health and decision quality

PRESENTER CUES · NOT SPOKEN

- Two separate hypothetical samples of 200
- Request success differs from safe solution quality
- Monitor service, behaviour, controls and human response capacity

SPOKEN SCRIPT · 192 WORDS

Evaluation and monitoring show the difference.

We already track availability, latency, load and request success. If 196 of 200 requests meet our service criteria, that ninety-eight percent may meet our chosen target.

But a completed request can contain a wrong solution.

With delegated judgment, we must evaluate the proposed path: is it correct for this problem, useful and safe? A plausible recovery plan could make an incident worse.

Consider a separate test of two hundred completed responses: 196 acceptable solutions, three incorrect and one dangerous. Again, ninety-eight percent. But the dangerous case violates our predefined release condition, so we block the release.

If it happens in production, stop the affected automation and switch to a safe fallback.

Functional correctness mattered before AI too. The added challenge is evaluating solutions constructed at runtime, before release and as conditions change.

If a human must recognize the problem or approve the response, monitor that dependency too: expertise, coverage, workload, response time and reported fatigue.

A person being online does not mean the control loop is working.

We need evidence that the system solves the problem safely, and that we can intervene when it does not.

SLIDE 09 / 09

Closing

PRESENTER CUES · NOT SPOKEN

- Two research directions and their systems lenses
- One homepage QR for articles and contact
- Sound engineering includes readiness for failure

SPOKEN SCRIPT · 99 WORDS

More code with unclear business value. Or a working demo with no viable operating model.

These are the problems behind my two public research projects. Both use systems thinking.

The Subprime Code Crisis examines SDLC imbalance through the theory of constraints.

Uncertainty Architecture examines delegated runtime judgment through control theory.

If you face these problems, I’d like to hear your experience. The QR code leads to my website, articles and contact details.

Let’s build the future we believe in. And let’s build it with sound engineering, ready to respond when things do not go as we hoped.

Thank you.

REFERENCE / NOT SPOKEN

Delivery and source notes

1,434 spoken words. Baseline R12: 1,423. Only the nine script bodies count. Cues, headings and references are excluded. At 80 words per minute, text takes about 17:56; allow another 2–3 minutes for pauses. Aim to finish speaking within 22 minutes of the 30-minute slot. These are planning estimates. Timed rehearsal and presenting-device checks remain pending.

The two 98% examples use separate hypothetical samples. Pause between them. Point to the Slide 6 formulas without reading them as equations. Preserve the recovery-drill conditions when tightening delivery.

NBER: Writing Code vs. Shipping Code. September 6, 2026. Figure 1 confirms 25.5x changed lines, 3.4x commits and 1.3x releases. Samples and tool stages differ. Async release effect is not separately estimated. These measures are not developer speed or ROI.

Chen and Stratton: Artificial Intelligence in the Firm. Current revision: 1 November 2026; 655 firms. Agent-adoption estimates: opened PRs +35.7%; resolved Jira issues +13.7%, borderline significant; review submission-to-approval elapsed time +56.1%. Observational identification assumptions apply. Elapsed time includes waiting; it is not active reviewer hours. Neither PR nor issue counts directly measure customer value.

Agarwal, He and Vasilescu. Version 2; 1,197 repositories including controls. Cognitive complexity: +34.85% / +42.87% (agent-first / IDE-first). Static warnings: +17.73% / +19.00%; the latter is not statistically significant. These are proxies, not direct defect or cost measures.

DORA 2025. Survey associations between AI adoption, higher throughput and lower stability. The visible slide retains n=4,867. Association does not establish causality.

GitClear: The Maintainability Gap. Vendor analysis of 623 million changes. Moved-code proxy -70% and duplication +81% compare 2023 with YTD2026. These structural signals do not isolate AI causality.

Martin-Lopez et al.: More Code, Less Understanding?. 69 participants, including 14 professionals. Short individual tasks do not directly measure whole-team ownership or production recovery.

Google SRE: Monitoring Distributed Systems. Operational metrics include latency, traffic, errors and saturation. The text already recognizes successful protocol responses with incorrect content.

NIST AI RMF: Measure. Evaluate behavior before deployment and in operation. Assess output quality, human oversight, operator competency and the ability to respond.

Slides 2-3 are illustrative systems models. The shared-context small-team example is illustrative, not a validated size threshold. Slide 8 has two separate hypothetical samples of 200: 196 requests meet service criteria in one; 196 solutions are acceptable, 3 incorrect and 1 critically harmful in the other. Both 98% figures are sample pass fractions, not confidence levels. Acceptance depends on the use case and predefined criteria. Conventional software already tests functional correctness; generated runtime solutions add a variable behavior space to evaluate and monitor.

“Thinking Systems” is the author’s proposed engineering category. Neither small happy-path demos nor a single passing test establish production readiness. Repeated model testing also needs representative contexts and cases, not only many runs of one prompt.

Bacchelli and Bird: Expectations, Outcomes, and Challenges of Modern Code Review. ICSE 2013; knowledge transfer and team awareness alongside defect detection. This was not an AI-adoption experiment.

The SDLC Is Out of Balance, author-review R10. The broader industry-level diagnosis and operating-model proposals are the author’s systems synthesis. Studies support parts of the mechanism, not measured industry-wide prevalence. A recovery drill supports only the scenarios and operating resources tested.

Chen-Stratton units: PR/Jira changes are per worker-month relative to baseline means. The review-time percentage uses elapsed days; it is not reviewer labour hours.